REQUIREMENTS & ARCHITECTURE WALKTHROUGH

GlowRoot

A cold-pressed juice brand, built as two products over one shared domain: a customer storefront, and the ops console that fulfils it.

One deck, five audiences. Architecture, UX, marketing, QA and DevOps each get a section — and one concrete ask at the end.

Product | Architecture | UI / UX | Marketing | QA | DevOps

`Turborepo + pnpm · Next.js 14 · TypeScript · storefront :3000`

HOW TO READ THIS DECK

Every role has a section — here is yours

Slides 3–11 are shared context; everyone should be in the room for those. From slide 12 the deck fans out by discipline, then converges again on status, roadmap and asks.

Slides	Section	Who it's for	What you get out of it
3 – 6	Problem, users, principles, scope	Everyone	Why this product exists, and what v1 deliberately excludes
7 – 11	Journey and the three key decisions	Everyone	The product logic the whole build hangs off
12 – 14	System shape, domain model, identity	Senior architects, security	The seams, the invariants, and the modelling debt
15 – 16	Design system and interaction contracts	UI / UX designers	Token set, component inventory, and the states we still owe
17	Positioning and funnel	Marketing	What the site does for acquisition, and the one known leak
18	Test strategy	QA	Critical paths, the status matrix, and what's mocked
19	Environments and Phase 4 runway	DevOps	What runs today, and what production actually needs
20 – 23	Status, roadmap, risks, asks	Everyone	Where we are, what's next, and what I need from you

THE PROBLEM

Perishable goods break the e-commerce default

CONSTRAINT 01

Juice is pressed to order

There is no warehouse buffer and no shipping window to hide behind — a bottle is made the night before it's drunk. So the delivery date isn't a fulfilment detail, it's part of the purchase, and it has to be settled before payment.

CONSTRAINT 02

Two revenue modes, one funnel

One-time buyers (trial, gifting) and subscribers have different economics, objections and journeys — but they must share one catalogue, one cart and one checkout, or we maintain the same flow twice.

CONSTRAINT 03

Ops needs tomorrow's answer

"What do we press tonight, and what produce do we buy?" A storefront can't answer that. Somebody was going to run it on a spreadsheet — which is exactly how order history and stock levels drift apart.

SO THE SHAPE IS

Two surfaces over one shared domain. A storefront that sells and an ops console that fulfils, reading the same products, the same orders and the same produce — so a sale on one side is visibly the same object as a press list on the other. That is the entire reason this is a monorepo and not two projects.

WHO WE'RE BUILDING FOR

Three users, three different definitions of success

ONE-TIME

Aditi · the gifter

Wants to try it, or send a box to someone. Zero appetite for commitment, and none for creating an account.

SUCCESS LOOKS LIKE

A frictionless first order. She should never meet a form field whose purpose she can't guess.

SURFACES THEY TOUCH

`Combos, gift boxes, build-your-own box, guest-friendly checkout`

RECURRING

Rohan · the subscriber

Drinks one most mornings. Wants it handled, but on his terms — his days, his cadence, pausable when he travels.

SUCCESS LOOKS LIKE

Set and forget, with control. He should be able to change his mind without emailing anyone.

SURFACES THEY TOUCH

`Plans, weekly/monthly cadence, start date + delivery days, account area`

INTERNAL

Meera · ops lead

Runs the press and the produce order. Today she'd be reconciling a spreadsheet against WhatsApp messages.

SUCCESS LOOKS LIKE

One screen that tells her what to press, what's moving, and what's running low.

SURFACES THEY TOUCH

`Orders list, status changes, KPI dashboard, materials + low stock`

PRODUCT PRINCIPLES

Four rules that settle most arguments for us

These are decisions, not aspirations. Each one is already enforced somewhere in the code — and each one has a cost we took knowingly.

01

Never gate the catalogue.

No pincode wall and no login wall in front of browsing. The landing page is the strongest marketing asset we have; gating it costs first impressions and search visibility.

Cost: people out of our area can browse things they can't buy — so the soft banner has to do real work.

02

Confirm serviceability before money, not before browsing.

One hard check, at checkout, inside the server action. Everywhere else, serviceability is only a hint.

Cost: a customer can fill an entire checkout form before being told no. Accepted — see slide 9.

03

Delivery date is a product decision.

Date and slot are a first-class, numbered checkout step — not a shipping estimate tucked onto the confirmation page.

Cost: checkout is longer than a standard cart. Worth it for something made the night before.

04

Every sale is traceable to the produce it consumes.

Each product carries a recipe: a bill of materials against the raw produce ops actually tracks.

Cost: the catalogue is more work to author. It's what makes the ops side more than a list of orders.

SCOPE

What v1 is — and what it deliberately is not

IN SCOPE — SHIPPED OR IN FLIGHT

- **Catalogue** 15 SKUs — 13 juices, 1 smoothie, 1 shot — plus seasonal produce and detail pages
- **Combos & boxes** curated combos, and a build-your-own box that keeps different fills as separate cart lines
- **Cart** drawer, quantities, survives reload
- **One-time checkout** pincode → date + slot → address → pay, with inline validation
- **Subscriptions** 3 plans, weekly/monthly cadence, start date and delivery days
- **Accounts** real passwords, server session, order history, order timeline
- **Public tracking** order ID plus email, no login required
- **Ops console** orders list and detail, status changes, KPI dashboard, materials with low-stock alerts

OUT OF SCOPE FOR V1 — NAMED, NOT FORGOTTEN

- **Recurring generation** a subscription writes one order today; nothing generates delivery two
- **Inventory deduction** recipes exist and are displayed, but stock never moves
- **Admin authentication** anyone who can reach the admin port can change any status
- **Real payment gateway** mocked deliberately, behind a swappable function
- **Notifications** no email, SMS or WhatsApp — confirmation is on-screen only
- **Waitlist capture** the out-of-area promise has nowhere to write
- **Customer-side cancel** cancelled exists as a status; only ops can set it
- **Catalogue editing** admin product and material pages are read-only views

Why say the out-of-scope list out loud: every one of those has a named owner and a slot in the Phase 4 sequence on slide 21. An unnamed gap becomes a production surprise.

THE TWO PURCHASE MODES

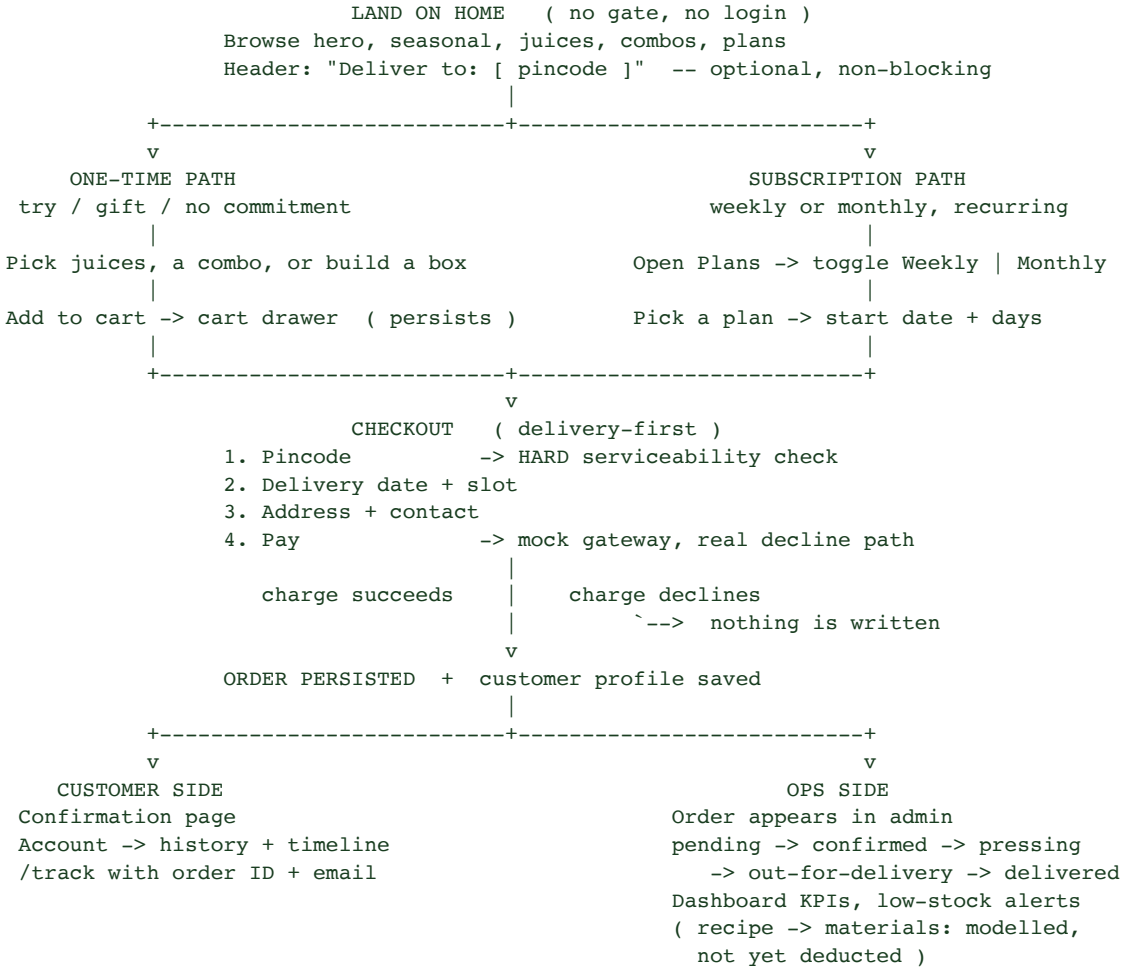
Parallel intents that converge on one checkout

	One-time	Subscription
Customer intent	Try it, or gift it. No commitment.	Handle my mornings. Recurring revenue for us.
Entry point	Juice carousel, combos, build-your-own box	Plans grid on the home page, or /subscribe
Configuration	Cart: pick bottles and quantities	Plan card: weekly | monthly, then start date and delivery days
Where cadence is chosen	n/a	On the plan card, before checkout — and never asked again
Price authority	Sum of cart lines, each snapshotting its own price	Server-side plan lookup. A client-supplied total is never trusted.
What checkout collects	Pincode, delivery date + slot, address, payment	Pincode, first delivery, address, payment + recurring consent
What it creates	An order with itemised lines	An order with a plan slug and no lines — see the gap on slide 20

The design constraint: both modes end in one checkout component running in two modes — not two checkouts. One place to get validation, serviceability and payment right, and one place where subscription-only fields must not leak into the one-time path.

END-TO-END JOURNEY

Landing to fulfilment, both paths



KEY DECISION 1 OF 3

Hybrid serviceability: soft while browsing, hard at checkout

Where	Behaviour
Home, catalogue, product pages	No check at all. Fully browsable by anyone.
Header pincode control	Optional and persisted. Shows a soft banner when out of area.
Add to cart	Allowed regardless of pincode.
Checkout / start subscription	**HARD check.** Enforced in the server action, not just the form.
Source of truth	One function, used by the header control, the form and the action alike.

WHY NOT A PINCODE GATE

A gate is the obvious answer and the wrong one: it kills the first impression and hides the entire catalogue from search. But juice is perishable and delivery-day dependent, so we genuinely can't take money for an address we can't reach. Hybrid is the only option that keeps both.

⚠ OPEN ISSUE — NEEDS A DECISION

The allow-list is currently switched off. A flag accepts every valid 6-digit pincode, so the real list — 10 pincodes across Mumbai, Bengaluru, Hyderabad and Kochi — the soft banner, and the whole hybrid contract are never exercised in practice. Before we flip it back, we have to decide what actually happens to an out-of-area visitor.

THE SUB-DECISION

- **Option A** capture email and pincode into a waitlist, then show the banner. Marketing gets a demand map by area.
- **Option B** soft-warn only and let them order anyway. Simpler, and we eat the failed deliveries.

KEY DECISION 2 OF 3

Checkout is delivery-first, because the product is perishable

1

Pincode

The one hard serviceability check. Fails closed.

2

Date + slot

A press-and-deliver lead time of one day; three morning slots.

3

Address

Prefilled from the saved profile on repeat orders.

4

Payment

Mocked gateway, with a real decline path and a real wait.

WHAT THIS BUYS US

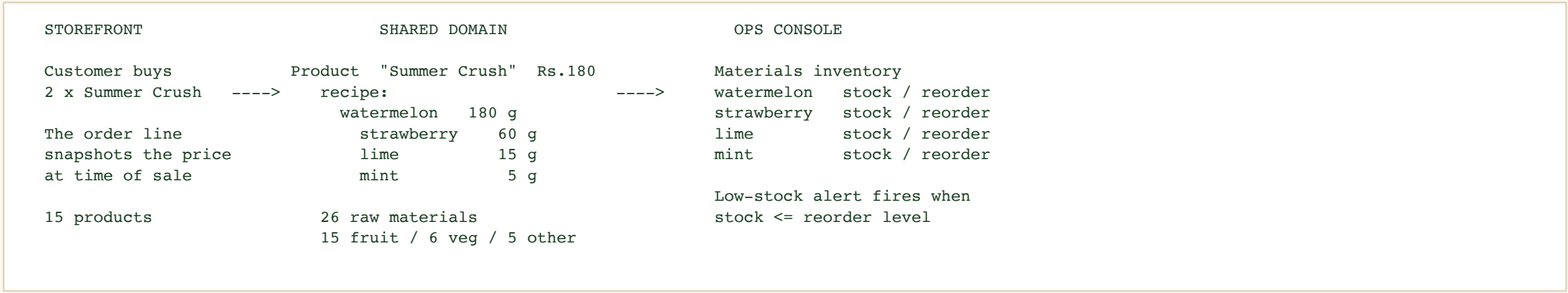
- **Nobody is told “no” after paying.** Serviceability and date both resolve before the charge.
- **A decline leaves nothing behind.** Payment runs before the order is written, so there is no half-order to reconcile later.
- **Ops gets a press date, not an estimate.** The delivery date is a committed field on the order rather than a computed guess.

WHAT'S STILL OPEN

- **The real cutoff rule.** Is it “order before 8pm → next day”? How far ahead can someone book? Which days don't we deliver at all?
- **Three sources currently disagree.** The form pins the date to today, the lead-time helper says tomorrow onwards, and plan config starts tomorrow. That's a live bug, not a nuance — and the cancellation window depends on the same rule.

KEY DECISION 3 OF 3

A bill of materials is what makes the ops side worth building



WHY IT MATTERS

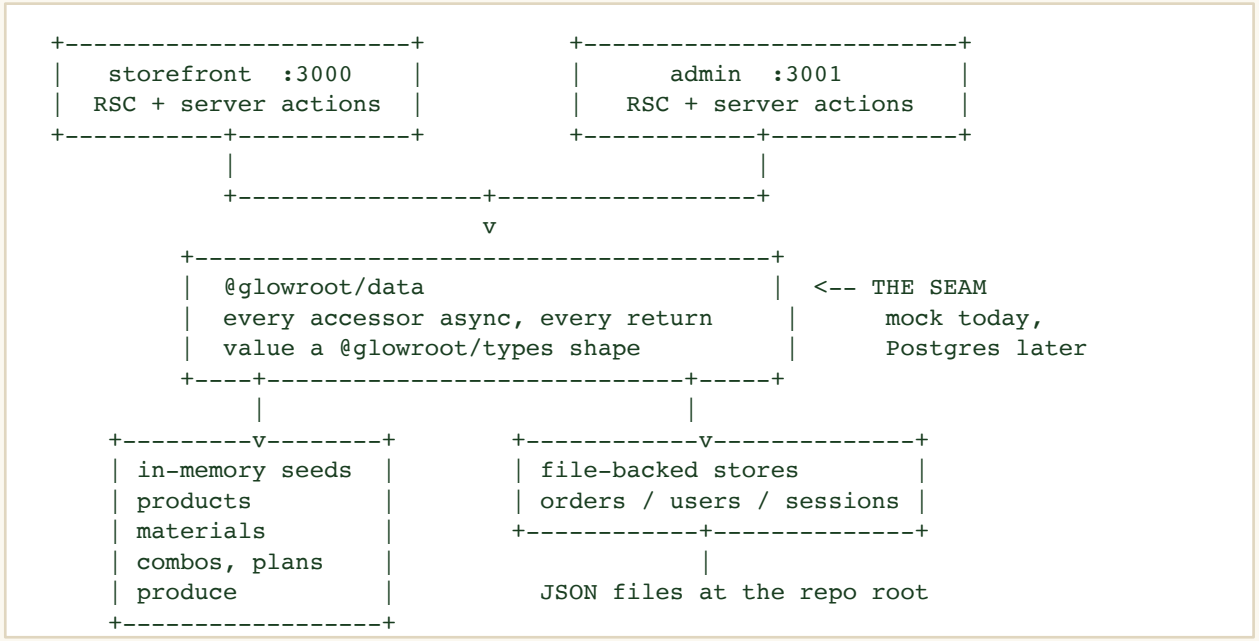
- **It closes the loop.** Without it, the ops app is a list of orders. With it, a sale connects to a purchase order for produce.
- **It's why this is one domain.** Both apps read the same product definitions, so a recipe change can never mean two different things.
- **It's modelled now, deliberately.** Getting the shape right before there's data to migrate is far cheaper than retrofitting it.

THE HONEST CAVEAT

- **Nothing decrements stock yet.** Recipes exist and are displayed; low-stock alerts run purely off seed values. The loop is drawn, not wired.
- **The fix is a ledger, not a counter.** Stock should become a sum over append-only movements — order, restock, wastage, correction — rather than a number we mutate.
- **Subscriptions can't compute it at all** until we decide whether subscribers pick their own bottles. See slide 16.

FOR SENIOR ARCHITECTS

System shape: two apps, five packages, one swappable seam



THE DELIBERATE CHOICE

There is no separate backend service. No Express app, no API process, no database. Server-side work happens in Next.js server actions, and the data layer is simply a package both apps import.

WHY THAT HOLDS UP

Because the accessor signatures are already the shape a real backend would expose — all async, all returning domain types. Swapping the bodies for database queries changes nothing above them. A standalone API only earns its keep when there's a non-Next consumer, and even then route handlers cover gateway webhooks fine.

WHAT I WANT YOUR EYES ON

Issue	Where it bites	My proposed answer
Read-modify-write JSON with no locking	Two concurrent checkouts and one is lost; a crash mid-write truncates the file	Postgres behind the same accessors — Phase 4 step 3
The data package exports fs and crypto from its root	One careless import from a client component breaks the build. Worse with a DB client in there.	Split into a client-safe catalogue entry and a server-only entry
No idempotency on order placement	Double-submit means two charges and two orders; the guard is client-side only	A unique request ID per attempt, inserted under a unique index

FOR SENIOR ARCHITECTS

Domain model: the invariants I want defended

THE SIX ENTITIES

- **Product** slug-keyed, and carries a recipe of raw materials
- **Material** raw produce, with a stock level and a reorder threshold
- **Combo / Bundle / Plan** curated sets, and the three subscription tiers
- **Order** customer, items, type, status, total, delivery date and slot, optional payment, append-only history
- **Payment** method, status, gateway reference, masked instrument hint
- **User / Session** email-keyed user; opaque server session with a hashed token

FOUR INVARIANTS ALREADY IN PLACE

- **Order lines snapshot their price.** A later catalogue price change cannot rewrite history.
- **Order history is append-only.** It's the audit trail and the customer-facing timeline in one object.
- **Only a masked instrument hint is stored.** Never a full card number or UPI handle.
- **The password hash never crosses the boundary.** A stripped public shape is the only user object a client can ever see.

THE MODELLING DEBT, WORST FIRST

Debt	Consequence
Email is the primary key	There is no stable user ID. Change an email and the order history detaches; every future foreign key has nothing to point at.
A subscription is not an entity	Cadence, start date and days are used to price and display, then dropped. Nothing generates delivery two; there's nothing to pause.
Stock is a mutable number	Not auditable and not idempotency-friendly. Should be a cached sum over an append-only movement ledger.
Order IDs are four random digits	A 9,000-value space with no uniqueness check on insert. Collisions are a matter of when, not if.
Status transitions are unconstrained	Any status can be set from any other. The lifecycle graph should be enforced server-side.

The one I most want fixed before there's real data: the missing user ID. Cheap now, expensive after launch.

FOR ARCHITECTS & SECURITY

Identity: what's real, and the one hole left

DONE

Passwords are real

Salted PBKDF2-SHA256 at 120,000 iterations with a 32-byte key, stored as salt plus digest. Plaintext never touches disk, and verification is constant-time with a length check first.

DONE

Sessions are server-side

Sign-in mints 32 random bytes. The hex goes to the browser in an HTTP-only, same-site cookie; only its SHA-256 digest is stored. Opaque, so it carries no claims — and revocable, which a signed token wouldn't be.

OPEN — BLOCKING

The admin app has no auth

Anyone who can reach the admin port can change any order's status. No login, no role check, and no record of who did what. This is the highest-priority item on the roadmap.

THE GUEST-TO-REGISTERED PATH — THE BIT WORTH UNDERSTANDING

State of an email	User record	Has credentials	Can check out
Unknown	none	—	no
Guest — ordered, never registered	exists	no	no, today
Registered	exists	yes	yes

Checking out without an account creates a passwordless record. Signing up later with the same email fills in the credentials rather than duplicating — so the customer's entire order history is there the moment they register.

TWO TRADE-OFFS I WANT SIGNED OFF, NOT INHERITED

- **Email enumeration is accepted.** Sign-in reveals whether an email is registered, so the UI can offer sign-up instead of a dead end. More informative than production auth usually is. The UX value is real; the mitigation is rate limiting, which we don't have yet.
- **Guest checkout contradicts itself.** The UI offers “continue as guest” and guest records get created, but placing an order rejects an unregistered email. Pick one. I'd allow real guest checkout — the machinery exists and it converts better.
- **Also missing entirely:** any password reset path, and rate limiting on sign-in.

FOR UI / UX DESIGNERS

The design system is inherited, not invented

The original hand-built prototype already had a coherent token system. It was lifted straight into a shared Tailwind preset, so both apps speak one design language from one file — the look didn't change, we just gained reusable parts.

COLOUR TOKENS

leaf	leaf.deep	leaf.soft	carrot	berry	apple	gold	cream	paper	ink	ink.soft	line
#2F5D34	#193D20	#5A8A4A	#D9772E	#7A4A8F	#D4533A	#E8B23F	#F6F0E2	#FBF8F0	#243018	#5B6450	#E0D6BF

TYPE & COMPONENT INVENTORY

- **Three type families** a display face for headings, a body face, and a script accent — all set in the shared preset
- **6 shared components** button, badge, card, eyebrow, product card, stats band — imported by both apps
- **33 storefront components** 10 are direct ports of prototype sections; the rest are the journey the prototype only hinted at
- **3 admin components** nav, status badge, status changer — intentionally thin
- **15 storefront routes, 5 admin routes** including build-your-own box, account order history, and public tracking

WHAT MUST BE INTERACTIVE ON THE CLIENT

- **Juice carousel** and the seasonal produce rings
- **Weekly / monthly billing toggle** cadence is chosen here, on the plan card, and never asked again
- **Cart drawer and fly-to-cart** cart and pincode both persist across reloads
- **Build-your-own box builder** two different fills of the same box stay separate cart lines
- **Checkout form** inline validation, gated on touch and submit rather than firing on every keystroke
- **Mobile drawer and shop mega-menu** two columns: seasonal, and juices + bundles

FOR UI / UX DESIGNERS

The states we owe, and the calls I need from you

STATES ALREADY HANDLED

- **Empty cart** in the drawer
- **Unknown order** on both the confirmation and tracking paths
- **Invalid pincode and inline field errors** at checkout
- **Payment decline** the mock gateway has a real failure path, deliberately triggerable from the UI
- **Order timeline** renders even for seed orders that predate history tracking

STATES STILL MISSING

- **Out-of-area** the contract promises “join the waitlist”; there is no waitlist, so there's nothing to design against yet
- **Logged-out header** the account control renders nothing when signed out, so the login page is reachable only by typing the URL
- **Subscription management** no pause, skip, swap or cancel screens — blocked on the model decision
- **Notifications** no confirmation email or delivery-day reminder to design

FIVE CALLS I NEED FROM DESIGN

Question	My leaning
Should a logged-out visitor be able to find the login page?	Yes — add the entry point. If hiding it was deliberate, let's write that down so it stops reading as a bug.
“Add to cart” or “add to order”?	Decide once and apply everywhere: button, drawer title, and the noun in all copy.
Cart drawer only, or a full /cart page too?	Drawer-first is fine, but a cart can't be deep-linked today and there's no full-page review step.
Do subscribers pick their own bottles?	The contract says mix and match, the form doesn't collect it, and the confirmation says we choose. This also decides whether ops can forecast produce for subscribers at all.
Should the soft banner escalate?	Today someone out of area can browse, fill the whole checkout, and only then be blocked. That's the intended hybrid — worth confirming it still feels right.

FOR MARKETING

What the site does for acquisition — and the one leak

WHAT THE ARCHITECTURE GIVES YOU

- **A fully crawlable catalogue.** No gate and no login wall. Every product, combo and seasonal page is server-rendered and indexable.
- **A balanced home page.** One-time and subscription get equal weight and cross-link, so a gifter can become a subscriber without starting over.
- **Seasonality as data, not copy.** Peak-season produce is modelled, so “what's in season now” can drive a campaign without a code change.
- **Pressed-to-order as the story.** Next-day delivery and a real press date are structural facts we can promise, not claims we have to soften.

MESSAGING THE PRODUCT CAN BACK UP

Pressed the night before · Delivered before breakfast · Peak-season produce · Three plans, from a starter to a family reset · Build your own box · Gift it without a subscription

⚠ THE LEAK

We promise out-of-area visitors a waitlist and capture nothing. Everyone who lands, wants it, and can't buy is lost silently — and we lose the demand-by-pincode map that would tell us which city to open next.

THE FUNNEL, AND WHERE IT BREAKS TODAY

Stage	Status	Gap
Land & browse	■ Built	None — this part is solid
Consider — detail, seasonal, combos	■ Built	None
Add to cart	■ Built	None
Check out	■ Partial	Requires an account; guest checkout is half-built
Confirm	■ Partial	On-screen only — no confirmation email
Retain / repeat	■ Partial	Order history exists; no reorder button
Subscribe & stay	■ Not built	No recurring delivery, no pause or skip
Out of area	■ Not built	No waitlist capture at all

What I need from you: the launch cities — which decides when we switch the pincode list back on — and whether a confirmation email is launch-blocking. I think it is.

FOR QA

Test strategy: the paths, the matrix, and what's mocked

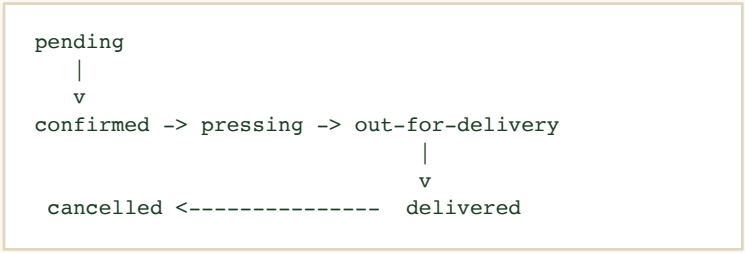
CRITICAL PATHS TO COVER

- **One-time order, happy path** browse → cart → all four checkout steps → order visible in admin
- **Subscription order** plan → cadence → config → checkout. Confirm the price comes from the server, not the form.
- **Payment decline** confirm nothing is written — no order, and no profile update
- **Guest → registered** order as a guest, sign up with the same email, confirm history attaches and no duplicate appears
- **Public tracking** order ID alone must fail; ID plus email must succeed
- **Cart persistence** reload mid-cart, and confirm two different box fills stay separate lines

NEGATIVE & EDGE CASES

- **Call the server actions directly.** The UI gate is a convenience; the action is the boundary. Every action must re-validate rather than trust the form.
- **Try to order as somebody else.** Identity now comes off the server session, so a request naming another email must be ignored, not honoured.
- **Double-submit checkout.** Expected to fail today — no idempotency key, so this should produce two charges. Log it; don't fix it in test.
- **Two concurrent orders.** Against the JSON store, one will be lost. Known, and the reason the database step exists.
- **Empty cart, invalid pincode, missing date.** All three must be rejected server-side, not only in the form.

THE STATUS MATRIX



Test that any status can be set from any status — **because today it can.** Transitions are unconstrained and the graph isn't enforced. Confirm each change appends to the order's history, and that the customer's timeline reflects it.

WHAT'S MOCKED — SO TEST THE SEAM

Payment is a stand-in that validates the instrument shape, waits about a second so “contacting your bank” is a real wait, and returns a reference. It already returns the exact shape the order persists — so test against that contract, not the fake. Then the real gateway swap is a body change with your suite still green.

Non-functional, and honest about it: there's no CI, no automated suite and no load testing today. The first things worth automating are the one-time happy path and the decline path — between them they catch most regressions across the four write paths.

FOR DEVOPS

What runs today, and what production actually needs

TODAY

Concern	Current state
Runtime	Two Next.js processes — storefront on 3000, admin on 3001
Build & tasks	A Turborepo pipeline over a pnpm workspace, with caching
Persistence	Two JSON files at the repo root, shared by both processes
Secrets	None — there's nothing to hold yet
CI / CD	None
Observability	None. No logs, no metrics, no error tracking.
Environments	Local only

⚠ THE GOTCHA THAT WILL COST SOMEBODY AN AFTERNOON

Cookies ignore port. In local development the storefront and the admin app share one cookie jar, so their sessions collide. The answer is distinct cookie names plus validating the session's role on read — never trusting the cookie name alone.

WHAT PHASE 4 WILL ASK YOU FOR

- **A managed Postgres per environment** plus a migration step in the deploy. The JSON stores go away.
- **Two deploy targets** one project per app; the monorepo makes this straightforward
- **Secrets management** gateway keys, the database URL, and the session configuration
- **A public webhook route** the gateway needs a callback endpoint — the first thing here that must be reachable from outside
- **A cookie-domain strategy** storefront and admin need separate cookie names and a real domain split, not just ports
- **A data migration window** the existing orders and users files need a one-shot import. Password hashes are portable, so nobody has to reset one.
- **Rate limiting on sign-in** per email and per IP. An in-memory window works to start but won't hold across instances — flag that when we scale out.
- **Backups and a restore drill** the first day we have data worth losing is the day Postgres lands

WHERE THE BUILD STANDS

Honest status, by area

Area	Status	Detail
Catalogue & browsing	■ Built	Home, products, detail pages, seasonal, combos, build-your-own box — all data-driven
Cart	■ Built	Drawer, quantities, persistence, fly-to-cart
One-time checkout	■ Built	Four steps, inline validation, order persisted, confirmation page
Payment	■ Partial	Mocked by design, with a real decline path behind a swappable seam
Serviceability	■ Partial	Built and wired, but the allow-list is switched off by a flag
Delivery-date rule	■ Partial	Three sources disagree — the form says today, the helpers say tomorrow onwards. A live bug.
Subscriptions	■ Partial	Plans, cadence and config work; one order is written and the recurrence is dropped
Accounts & identity	■ Built	Real passwords, server sessions, order history, timeline, public tracking
Ops console	■ Built	Orders list and detail, status changes, KPI dashboard, materials with low-stock alerts
Admin auth	■ Not built	None at all. Top of the roadmap.
Inventory movement	■ Not built	Recipes are modelled and displayed; stock never changes
Notifications	■ Not built	On-screen confirmation only

Read this as the deck's credibility test. Nine of the twelve rows are green or amber, and the three reds are all named on the roadmap. Nothing on this slide is a surprise to me — that is the whole point of having it.

ROADMAP

Phase 4: the sequence, and why this order

#	Step	Why it goes here	Rough size
1	Admin authentication and role gating	The only outright security bug left. Needs no database — it works against the current stores, and the customer-side session work already proves the pattern.	~1 day
2	Split the data package's export surface	Client-safe catalogue versus server-only stores, so misuse fails at build time. Cheap, and a prerequisite for a DB client living in that package.	~1 hour
3	Postgres behind the same accessors, plus migration scripts	Fixes concurrent-write loss and the truncation risk in one move. The accessor signatures don't change, so no app code does either.	~2–3 days
4	Idempotency key, real order sequence, one transaction	Falls out naturally once there's a database. Kills the double-charge and the ID collision together.	~half day
5	Subscription entity and inventory ledger	The two genuinely new domain models. Unblocks pause/skip/cancel, recurring deliveries, and produce forecasting.	~2 days
6	Real gateway, password reset, rate limiting	Independent of each other — slot them in as needed once the foundation is there.	as needed

The sequencing argument: steps 1 and 2 come before the database on purpose. Sessions change the accessor signatures, and I'd rather change them once against JSON files than twice against Postgres. Doing the security fix first also means we are never knowingly running an unauthenticated admin panel against a real database.

RISKS & OPEN DECISIONS

Six things that need an owner and a date

Risk / decision	Impact if we do nothing	Call needed from	My recommendation
Admin panel is unauthenticated	Anyone reaching the port can alter any order. Blocks any real launch.	Architecture + me	Fix it first, before the database. Roles rather than a shared password, and record who changed what.
A subscription is not an entity	No second delivery is ever generated, there's nothing to pause, and ops can't tell what to press for a subscriber.	Me + architecture	Build the real entity that generates an order per delivery. Get the shape right before there's data to migrate.
The delivery-date rule contradicts itself	Subscription checkout is internally inconsistent today, and it's the rule cancellation windows depend on.	Me + ops	Define the cutoff — “order before 8pm → next day” — then make one helper the single source of truth.
Serviceability is switched off	The hybrid contract is never exercised, and out-of-area demand is lost silently.	Marketing + me	Decide the launch cities and the out-of-area experience, add waitlist capture, then flip the flag back.
Guest checkout contradicts itself	The UI offers it and the server rejects it. Measurable conversion loss.	Design + me	Allow real guest checkout, with claim-by-email on signup. The machinery already exists.
No confirmation email	Customers have no record of an order once they close the tab.	Marketing + DevOps	Treat it as launch table stakes. Order placement is the natural hook.

WHAT I NEED FROM YOU

One ask each — and the date I need it by

Senior architects	Review the three issues on slide 12 and tell me whether “no separate service” still holds at ten times the traffic. Then rule on the missing user ID — I want that fixed before there's data to migrate.
UI / UX designers	The five calls on slide 16 — especially whether subscribers pick their own bottles, because that one decides whether ops can forecast produce at all. Plus the out-of-area and logged-out states we have nothing for.
Marketing	Launch cities, so I know when to switch the pincode allow-list back on. And a yes or no on the confirmation email being launch-blocking — I believe it is.
QA	Automate the one-time happy path and the payment decline path first. Then treat the documented gaps on slide 18 as expected failures rather than new bugs, and log them against roadmap items.
DevOps	A cost and lead time for managed Postgres plus two deploy targets, and the shape of secrets management. The cookie-port collision on slide 19 is the one thing I'd flag before you start.

Every open question in this deck is written down in the repo docs — none of it lives only in my head.